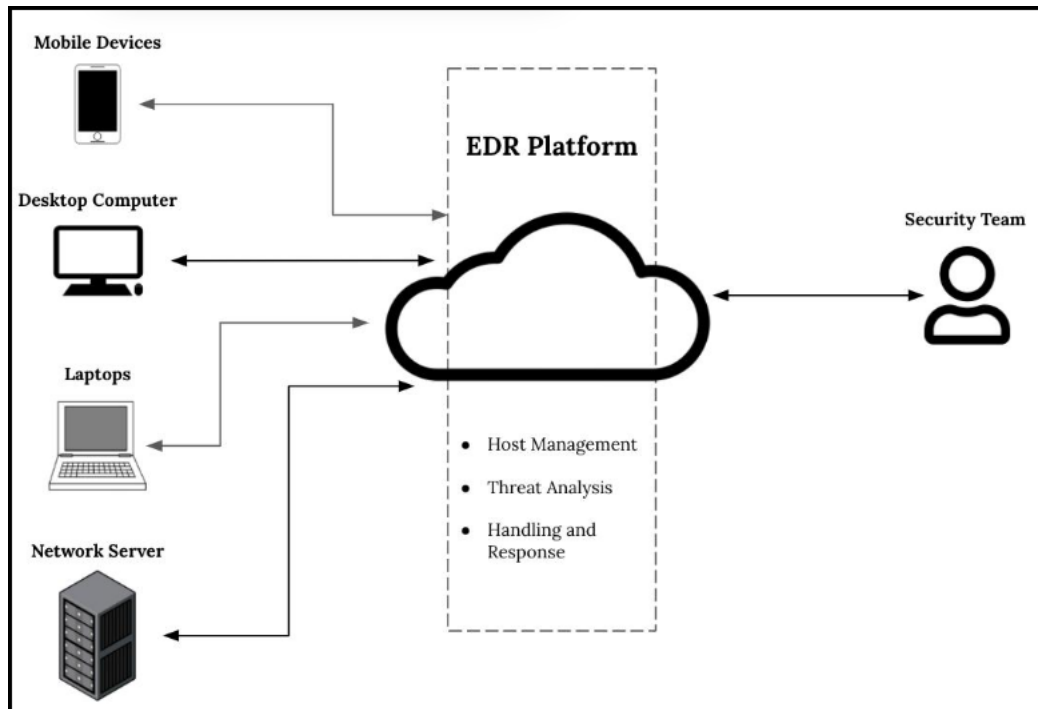
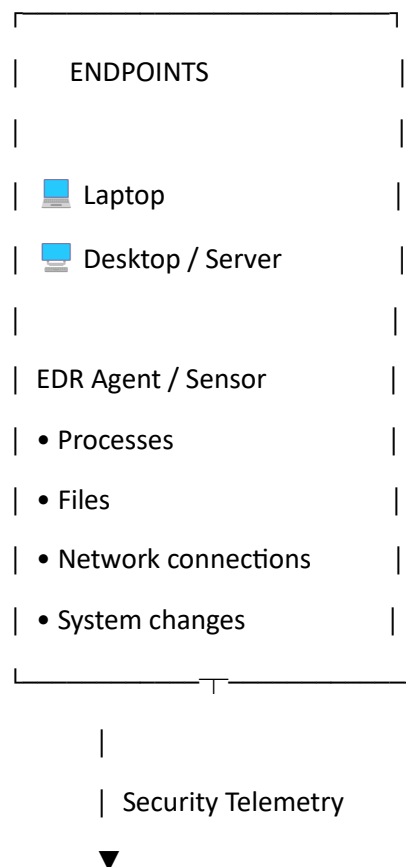


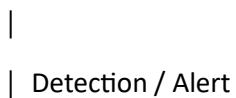
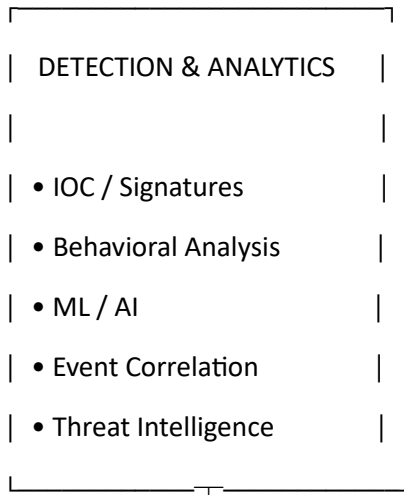
1. Generic EDR Architecture

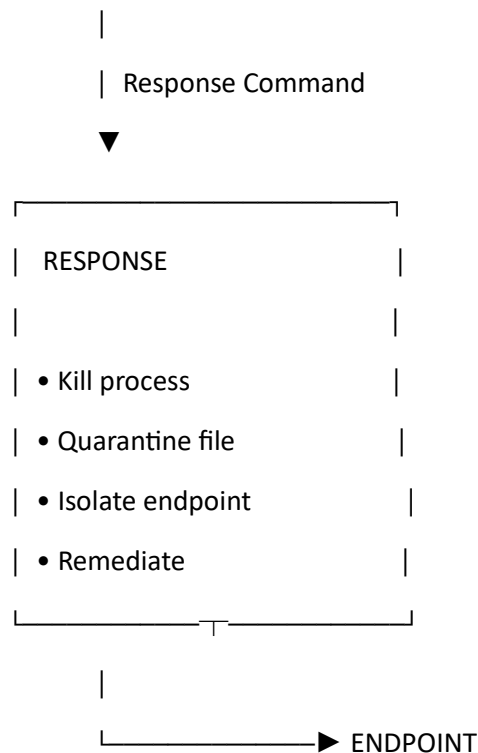
How a typical EDR works



Architecture Flow







How the flow works

1. Endpoint monitoring

An EDR agent runs on each laptop, desktop, server, or other protected endpoint. It continuously observes security-relevant activity such as processes, files, system changes, and network connections.

2. Data is sent to the central platform

The agent sends relevant telemetry to the EDR's backend, which is commonly **cloud-hosted**. The platform collects, organizes, enriches, and stores this information so it can be analyzed and investigated later.

3. Detection engine analyzes the activity

The platform looks for threats using multiple techniques such as known indicators (IOCs), behavioral rules, machine learning, threat intelligence, and correlation of related events. Some detection and prevention can also happen directly on the endpoint.

4. A threat becomes an alert/incident

When suspicious or malicious activity is identified, the EDR turns the relevant events into an **alert or incident**, usually adding context such as severity, affected device, processes involved, and the attack sequence.

5. Alert appears on the security dashboard

Security analysts see the alert in the EDR console. The dashboard allows them to investigate the incident, view timelines and affected endpoints, and understand what happened.

6. Response is performed

The analyst—or an automated response mechanism—can take action such as **isolating the**

endpoint, killing a malicious process, quarantining a file, or performing remediation. The response command can then be sent back to the endpoint through the EDR agent.

In one line

Endpoint → Collect Telemetry → Cloud/Backend → Detect & Correlate → Alert → Security Dashboard → Investigate → Respond → Endpoint

2. How We Will Build AiBoO

2.1 The Basic Idea

AiBoO will be built as a **collection of specialized security components rather than one large program.**

Each component has one job:

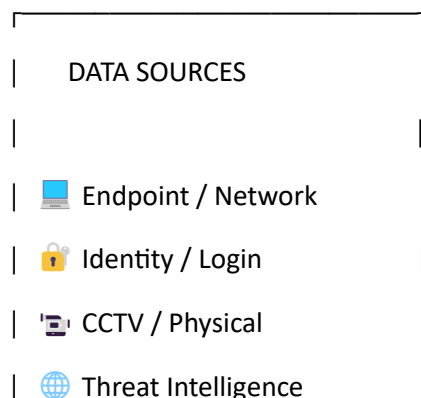
- **Sensors** collect security events.
- **Event Bus** carries those events to the right components.
- **Specialist Agents** analyze different types of events.
- **Correlation Engine** connects findings from different agents.
- **Response Engine** decides and executes security actions.
- **Backend** stores and manages the data.
- **Dashboard** shows the final alerts and system status to the security team.

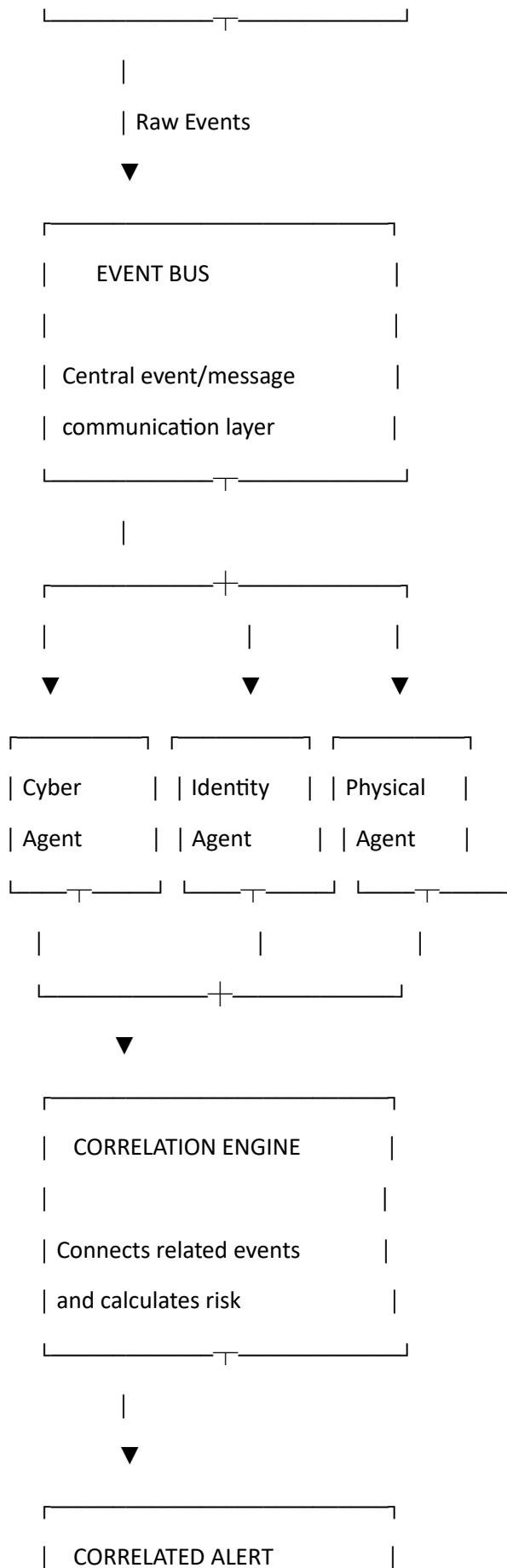
The proposed design uses Python for the security-agent layer, while the existing application can use a MERN backend. The two systems communicate through APIs or a message broker rather than being mixed together.

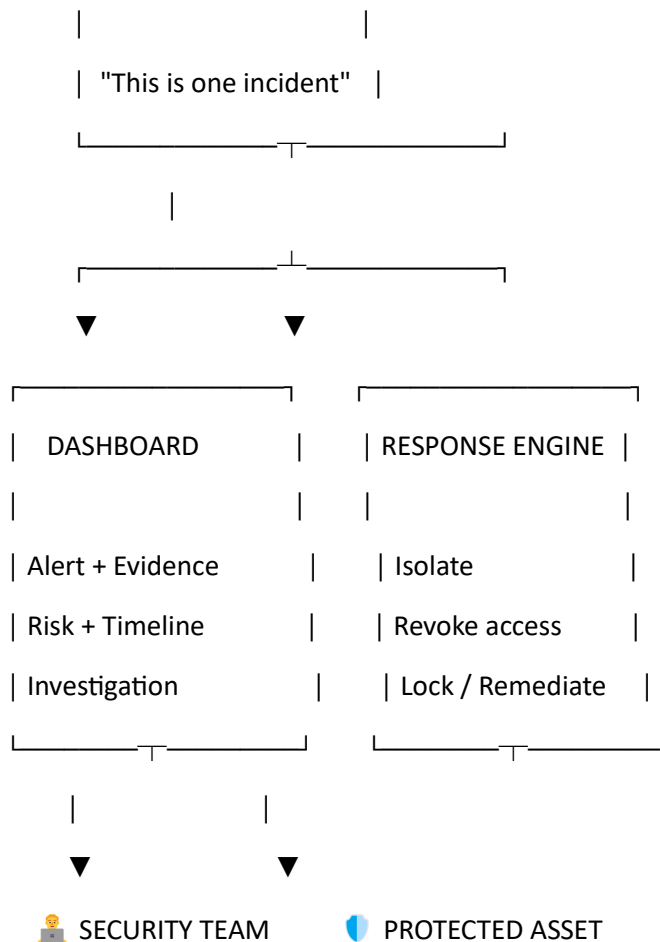
2.2 Overall Architecture

In this design, the central Event Bus plays the same basic role as the message broker: one event can be received by multiple independent services without those services being directly connected to each other. This is a standard event-driven architecture pattern.

Proposed AiBoO flow







2.3 What Happens From Start to Finish?

Step 1 — Something happens

Everything starts with an **event**.

For example:

A user logs in from a new location.

Or:

A computer suddenly starts transferring a large amount of data.

Or:

CCTV detects someone entering a restricted area.

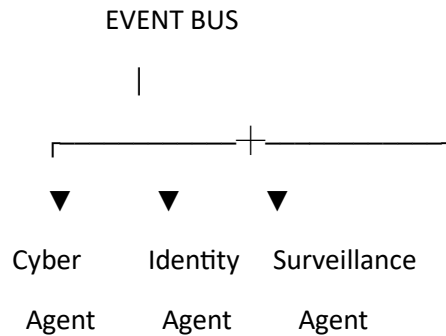
The event is converted into a standard format called a **ThreatEvent** and sent to the Event Bus. This matches the proposed architecture where raw sensor signals become ThreatEvent objects.

Step 2 — Event Bus distributes the event

Think of the Event Bus as a **post office**.

The event comes in once, and the bus delivers it to whichever agents are interested in it.

For example:



This means the Cyber Agent doesn't need to directly communicate with the Identity Agent.

That makes the system easier to expand later. A new agent can subscribe to the events it needs without rewriting the existing agents. This producer → broker → consumer model is a standard event-driven architecture pattern.

2.4 The Specialist Agents

Instead of making one huge AI system responsible for everything, we divide the work.

Cyber Threat Agent

Looks at things such as:

- Network activity
- Suspicious processes
- Malware behavior
- Unusual data transfers
- Lateral movement

Identity Agent

Looks at:

- Login attempts
- Authentication failures
- Device identity
- Location
- Impossible travel
- Identity mismatches

Physical/Surveillance Agent

Looks at:

- CCTV events
- Badge access
- Restricted-zone access
- Face/badge mismatch
- Physical anomalies

The original implementation plan specifically proposes Cyber Threat, Identity Verification, Surveillance, and Pseudo-Lock agents.

2.5 What Does an Agent Actually Return?

An agent shouldn't simply say:

"Something looks suspicious."

It should produce a **structured finding**.

For example:

AgentFinding

Event:

User login detected

Agent:

Identity Agent

Finding:

Suspicious login

Confidence:

82%

Reason:

New device + unusual location + unusual time

Recommended Action:

Step-up verification

The proposed design calls this output an **AgentFinding**, containing the finding, confidence score, and recommended actions.

This is important for the developer because **every agent should follow the same input/output format.**

2.6 The Correlation Engine — The Most Important Part

This is where AiBoO becomes more than a collection of individual security tools.

10:01 AM → Suspicious login

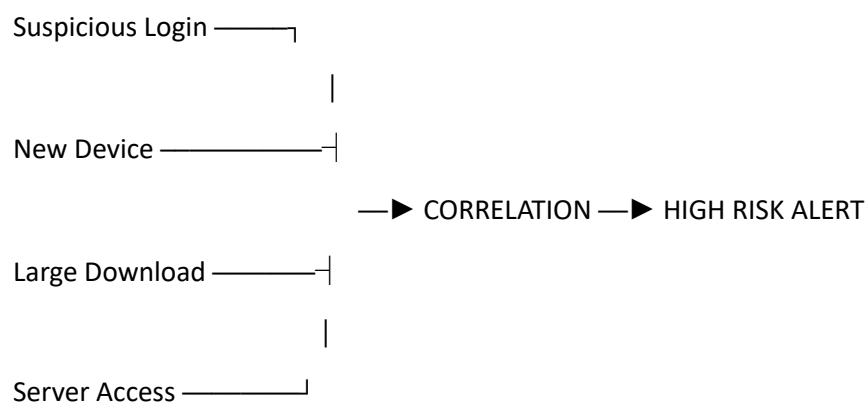
10:02 AM → New device detected

10:03 AM → Large file download

10:04 AM → Restricted server accessed

Each event individually might not be enough to trigger a serious response.

The **Correlation Engine** connects them.



The current proposed design uses a **30-second sliding window** to look for related findings and generate one CorrelatedAlert instead of treating every finding as a completely separate incident.

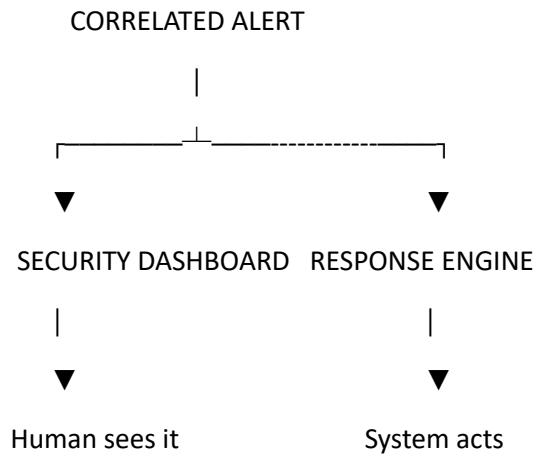
Simple way to understand it:

Agents find the pieces.

Correlation Engine puts the pieces together.

2.7 What Happens After an Alert?

Once the Correlation Engine creates a CorrelatedAlert, **two things happen at the same time:**



The proposed architecture explicitly sends the correlated alert to both the dashboard and the autonomous response engine.

Dashboard

The dashboard shows:

- What happened
- Who/what was affected
- Severity
- Confidence
- Timeline
- Evidence
- Recommended action
- Actions already taken

Response Engine

Depending on the risk, it can:

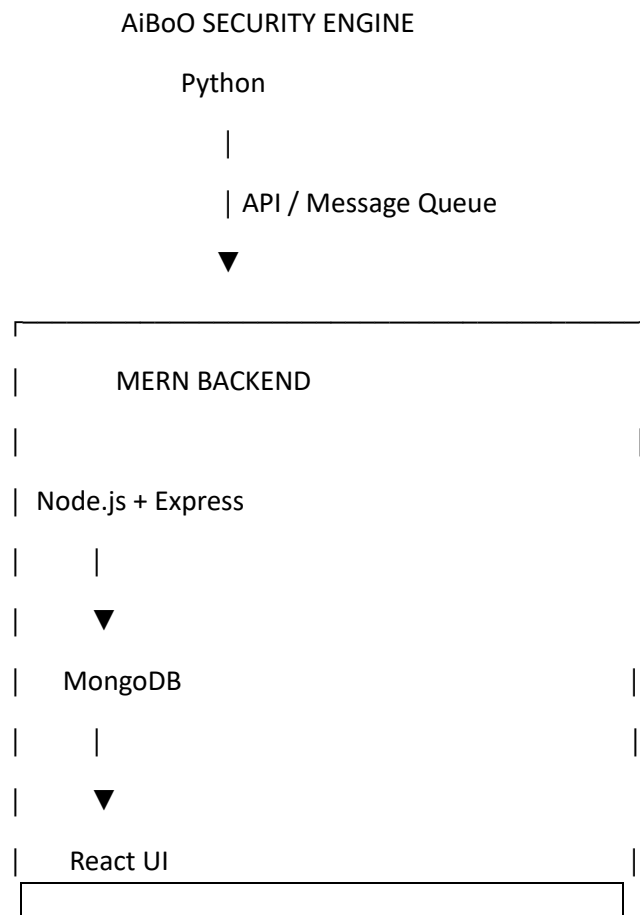
- Isolate a device
- Revoke identity/session access
- Lock a physical zone
- Quarantine something
- Notify the security team
- Trigger a predefined response workflow

The proposed Layer 3 architecture is specifically intended to convert intelligence into containment, isolation, lockdown, and mitigation actions.

2.8 How This Connects to the MERN Backend

Do NOT try to put the Python agents inside the MERN application.

Keep them as a separate security service.



The earlier implementation plan proposes exactly this separation: the Python agent system and MERN backend remain separate processes and communicate through a defined interface.

For the MVP, the simplest connection is:

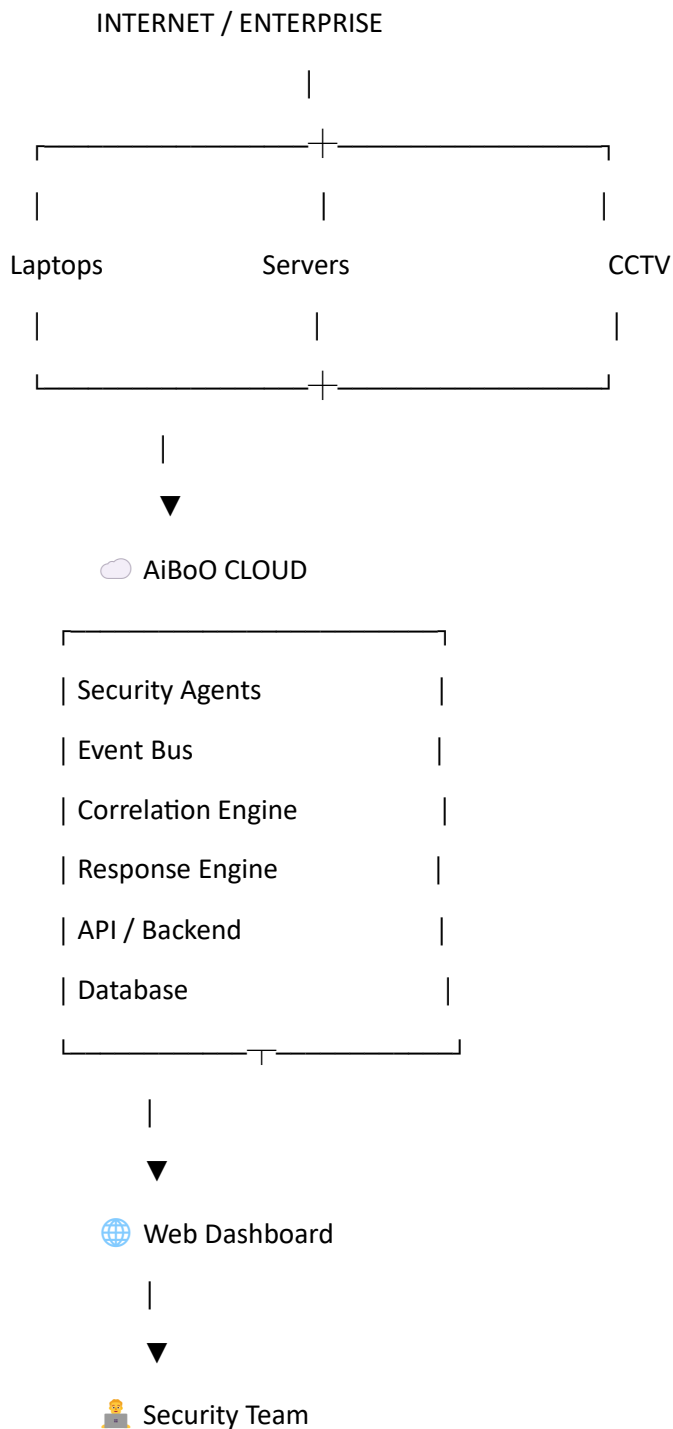
Python → HTTP API → Express → MongoDB → React

For a more scalable production system, **Redis/RabbitMQ can sit between the Python security engine and Node.js**, allowing events to be queued and processed independently.

2.9 Where Does the Cloud Come In?

The cloud is basically where the **central AiBoO platform runs**.

A simplified deployment could look like:



2.10 Do We Need an LLM?

No — not for the core detection system.

This is something I would explicitly mention in the PDF because it prevents the developer from assuming that "AI-powered" means everything needs an LLM.

The current design uses deterministic logic such as:

- Rules

- Thresholds
- Pattern matching
- Risk scoring
- Correlation

The source material explicitly states that the core system does **not require an LLM**.

An LLM can be added later for things like:

"Explain this incident in simple English."

"What happened during this attack?"

"Summarize the evidence for the SOC analyst."

So:

CORE SECURITY

Rules + Detection + Correlation + Risk Scoring



Human-readable explanation

2.11 The Complete AiBoO Flow

This is the **one diagram I would put prominently in the PDF**:

